

LeetCode編碼：

234

系所：北聯(北醫) 生物醫學工程學系

學號：B812111001

姓名：張博翔





給定一條 Singly Linked List，判斷其節點從頭到尾讀與反向讀是否完全相同，判斷是否為 Palindrome。

由於 Singly Linked List 的節點只知道下一個節點在哪，無法往回，使同時從兩端比對複雜許多。

- 以 $1 \rightarrow 2 \rightarrow 2 \rightarrow 1$ 為例，正向讀與反向讀皆是 $[1, 2, 2, 1]$ ，兩者相同，為 Palindrome，回傳 True。當串列只有一個節點時單一元素就是 Palindrome，也是直接回傳 True
- 以 $1 \rightarrow 2 \rightarrow 3$ 為例，正向是 $[1, 2, 3]$ ，反向是 $[3, 2, 1]$ ，回傳 False。
- 需要注意當串列長度為奇數時，中間節點不需比對。
- 反轉後半段會暫時破壞原始結構，比對完後需再將後半段還原。



- 解法一：陣列轉換 + 雙指標
- 出發點是解決 Singly Linked List 只能單向走的限制。把所有節點依序存入陣列，轉換成可以用索引存取的結構，再用雙指標從兩端往中間逐一比對是否相同。

好處是邏輯直覺。缺點是需要額外開一個與串列等長的陣列，空間複雜度為 $O(n)$ ，當串列非常長時記憶體消耗大，且該空間並非演算法的需求，只是為了彌補不能隨機存取的缺陷。



- 解法二：快慢指標 + 反轉後半段空間
 - 不須額外陣列的前提下，直接在Linked List完成回文判斷，空間複雜度降至 $O(1)$ ，過程分三個步驟。
 - 第一步找到串列中點。使用 Floyd 's Tortoise and Hare：
因為不知道Linked List的長度，沒辦法直接除以 2 定位中點，因此使用快慢指標，slow 每次走一步，fast 每次走兩步，當 fast 走到尾端時，slow 恰好在中點。
奇數長度時 slow 停在正中央的節點，偶數長度時 slow 停在前半段的最後一個節點 slow.next 都是後半段的起點。
 - 第二步反轉後半段。從 slow.next 開始用迭代方式將後半段的指標方向反轉，從後半段的尾端往前比對，Singly Linked List無法往回走，反轉後就能正向走，等同於從後往前讀原串列。
 - 第三步雙指標比對。一個指標從串列頭部出發，另一個從反轉後的後半段頭部出發，若對不上回傳 False。
比對完後將後半段再反轉一次還原結構，確保原始串列不被破壞。
- 優點是 $O(1)$ 空間，不需要任何額外記憶體。缺點是流程長，且反轉操作會暫時破壞串列，需要特別注意還原。



- 難在Singly Linked List只能往前走，但Palindrome判斷需要從兩端往中間比對。
陣列法用空間換取了隨機存取的能力；快慢指標加反轉則是直接改變後半段的走向，讓兩個指標都能向前走達到從兩端夾擠的效果。
快慢指標這個技巧，在 **LC 141**、**LC 876** 等題也反覆出現，是**Linked List**的重要技巧。

方法	Time	Space
陣列轉換	$O(n)$	$O(n)$
遞迴	$O(n)$	$O(n)$
快慢+反轉	$O(n)$	$O(1)$



```

1 class Solution(object):
2     # 解法一：陣列轉換 + 雙指標
3     def isPalindrome_array(self, head):
4         vals = []
5         node = head
6         while node:
7             vals.append(node.val)
8             node = node.next
9         left, right = 0, len(vals) - 1
10        while left < right:
11            if vals[left] != vals[right]:
12                return False
13            left += 1; right -= 1
14        return True
15

```

```

16 # 解法二：快慢指標 + 反轉後半 O(1) 空間
17 def isPalindrome_reverse(self, head):
18     if not head or not head.next:
19         return True
20     slow = fast = head
21     # fast 每次走兩步，slow 每次走一步
22     # 當 fast 到尾端時，slow 恰好在中點
23     # 條件是 fast.next 和 fast.next.next 都存在
24     # 確保 fast 還能再走兩步，避免在偶數長度時 slow 超過中點
25     while fast.next and fast.next.next:
26         slow = slow.next
27         fast = fast.next.next
28
29     def reverse(node):
30         prev = None
31         while node:
32             nxt = node.next
33             # 先存下一個節點，因為反轉後就找不到了
34             node.next = prev
35             prev = node
36             node = nxt
37         return prev
38
39     # slow.next 是後半段的起點
40     # 奇數長度時 slow 在正中央，中央節點不需比對
41     # 偶數長度時 slow 在前半段最後一個，slow.next 是後半段第一個
42     second = reverse(slow.next)
43     p1, p2, result = head, second, True
44     while p2:
45         if p1.val != p2.val:
46             result = False
47             break
48         p1 = p1.next
49         p2 = p2.next
50     # 比對完後還原後半段，避免破壞原始串列結構
51     slow.next = reverse(second)
52     return result
53

```

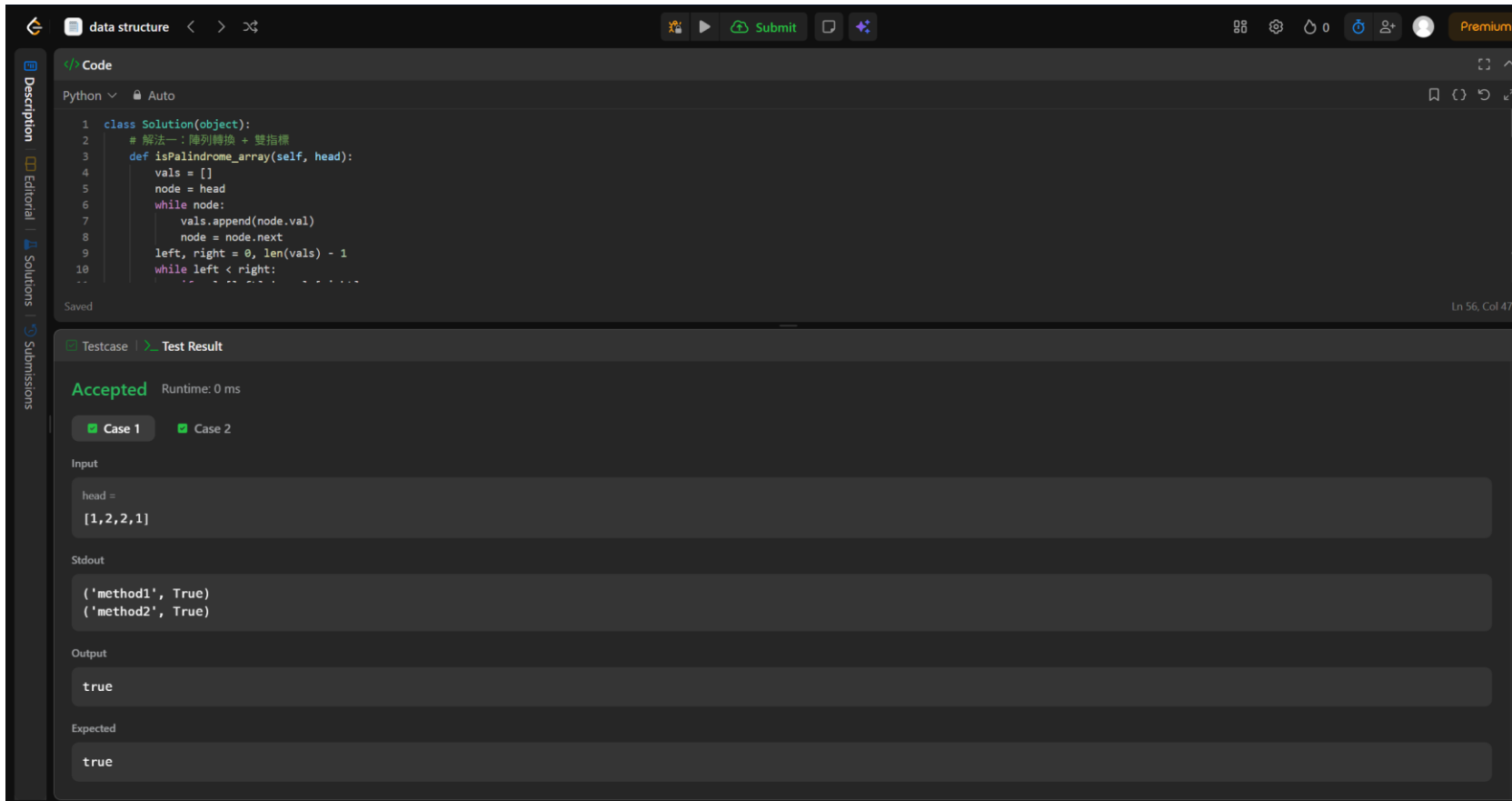
```

54 def isPalindrome(self, head):
55     res1 = self.isPalindrome_array(head)
56     res2 = self.isPalindrome_reverse(head)
57
58     print("method1", res1)
59     print("method2", res2)
60
61     return res1
62

```



- 結果證明：



```

1 class Solution(object):
2     # 解法一：陣列轉換 + 雙指標
3     def isPalindrome_array(self, head):
4         vals = []
5         node = head
6         while node:
7             vals.append(node.val)
8             node = node.next
9         left, right = 0, len(vals) - 1
10        while left < right:
11            ...

```

Accepted Runtime: 0 ms

Case 1 Case 2

Input

head =
[1,2,2,1]

Stdout

('method1', True)
('method2', True)

Output

true

Expected

true